Full Name: Vuong Hai Anh
Student ID: 11247142
Class: Data Science 66B

Hands-on Lab: Securing dvdrental

1. Objective

The objective of this lab is to improve the security of the dvdrental PostgreSQL database by implementing Role-Based Access Control (RBAC) and Row-Level Security (RLS).
The lab focuses on two main goals:
1. Create a restricted cashier role and assign appropriate privileges based on the Principle of Least Privilege.
2. Apply Row-Level Security to ensure that each cashier can only access payment transactions associated with their own staff_id.
Before configuring the security mechanisms, the existing staff table was checked to identify the available staff accounts.

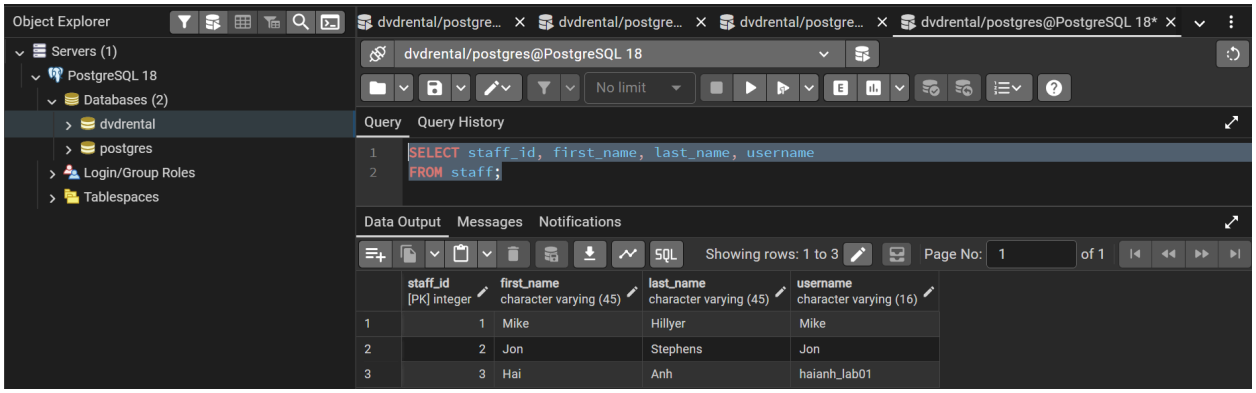


Figure 1. Existing staff accounts in the dvdrental database.
The result shows that Mike has staff_id = 1 and Jon has staff_id = 2. These staff records are later used to determine which payment rows each cashier is allowed to access.

2. Lab 1: Implementing RBAC

2.1. Creating the cashier role and user accounts

A group role named cashier_role was created to represent the common permissions required by store cashiers.
Two PostgreSQL user accounts, mike and jon, were then created and assigned to this role.
The main commands were:
CREATE ROLE cashier_role;

CREATE USER mike WITH PASSWORD 'mike123';
CREATE USER jon WITH PASSWORD 'jon123';

GRANT cashier_role TO mike;
GRANT cashier_role TO jon;

The role membership was verified using PostgreSQL system catalogs.

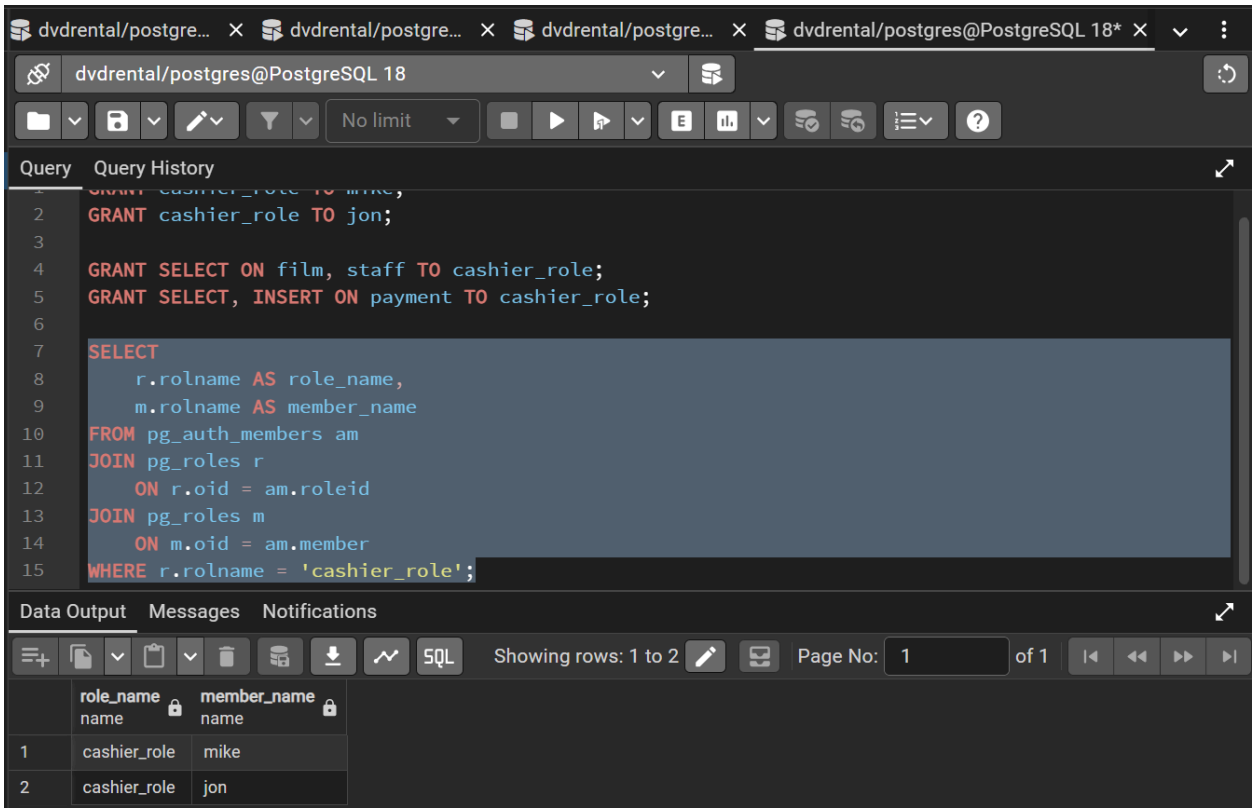


Figure 2. Mike and Jon were successfully assigned to the cashier_role.

The result confirms that both users inherit privileges granted to cashier_role.

2.2. Granting table privileges

Following the Principle of Least Privilege, the cashier role was granted only the permissions necessary to perform normal cashier operations.

GRANT SELECT ON film, staff TO cashier_role;
GRANT SELECT, INSERT ON payment TO cashier_role;

The assigned privileges were verified using information_schema.role_table_grants.

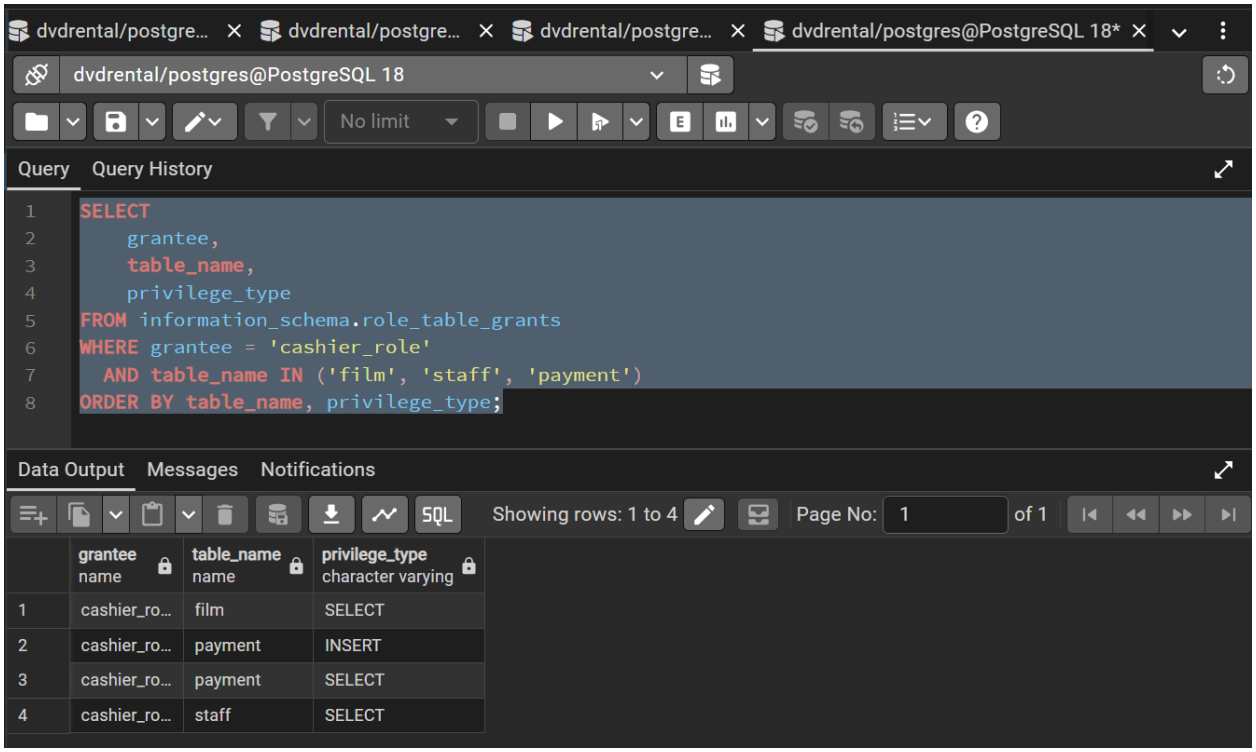


Figure 3. Table privileges granted to the cashier_role according to the Principle of Least Privilege. The result shows that cashiers can read the film and staff tables and can read and insert records into the payment table. No unnecessary UPDATE or DELETE privilege was granted.

2.3. Testing restricted privileges

To verify that unauthorized operations were blocked, the session was switched to the mike role and a DELETE operation was attempted on the payment table.

```
SET ROLE mike;

DELETE FROM payment
WHERE payment_id = -1;
```

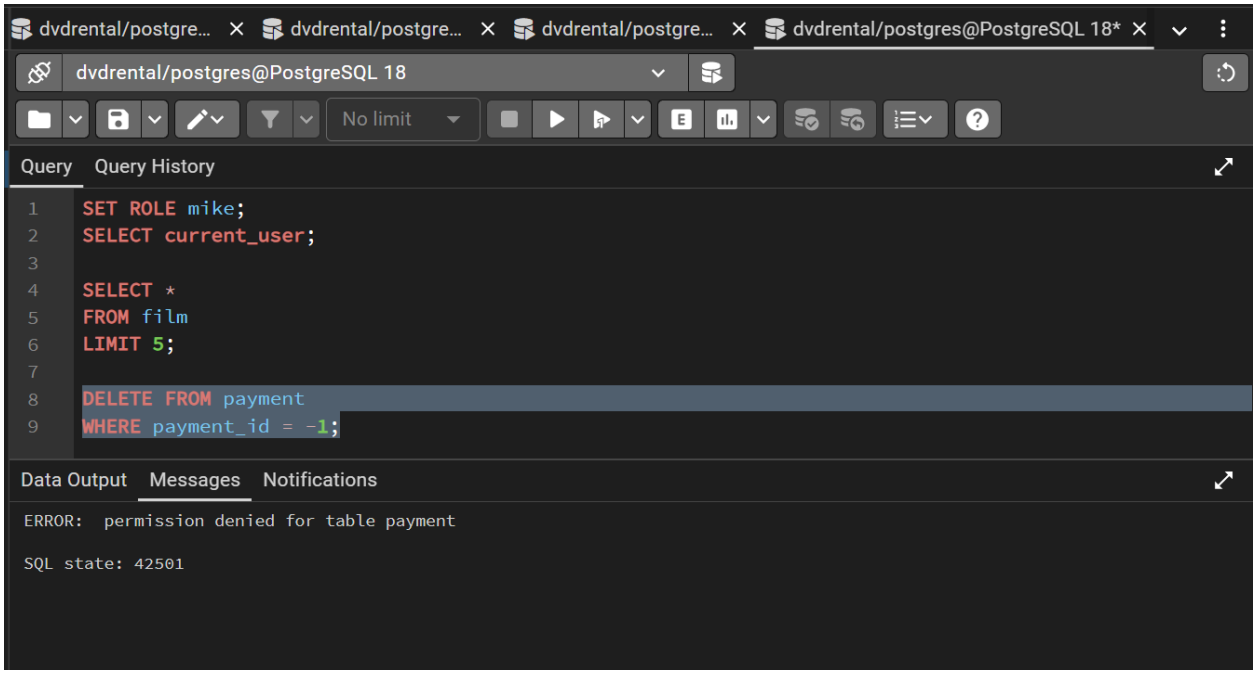


Figure 4. The cashier account cannot delete payment records because DELETE privilege was not granted. PostgreSQL returned:
ERROR: permission denied for table payment
This demonstrates that RBAC successfully prevents the cashier account from performing operations outside its assigned responsibilities.

3. Lab 2: Implementing Row-Level Security

3.1. Examining payment records before RLS

Before enabling Row-Level Security, the number of payment records handled by each staff member was checked.

```
SELECT staff_id, COUNT(*) AS number_of_payments
FROM payment
GROUP BY staff_id
ORDER BY staff_id;
```

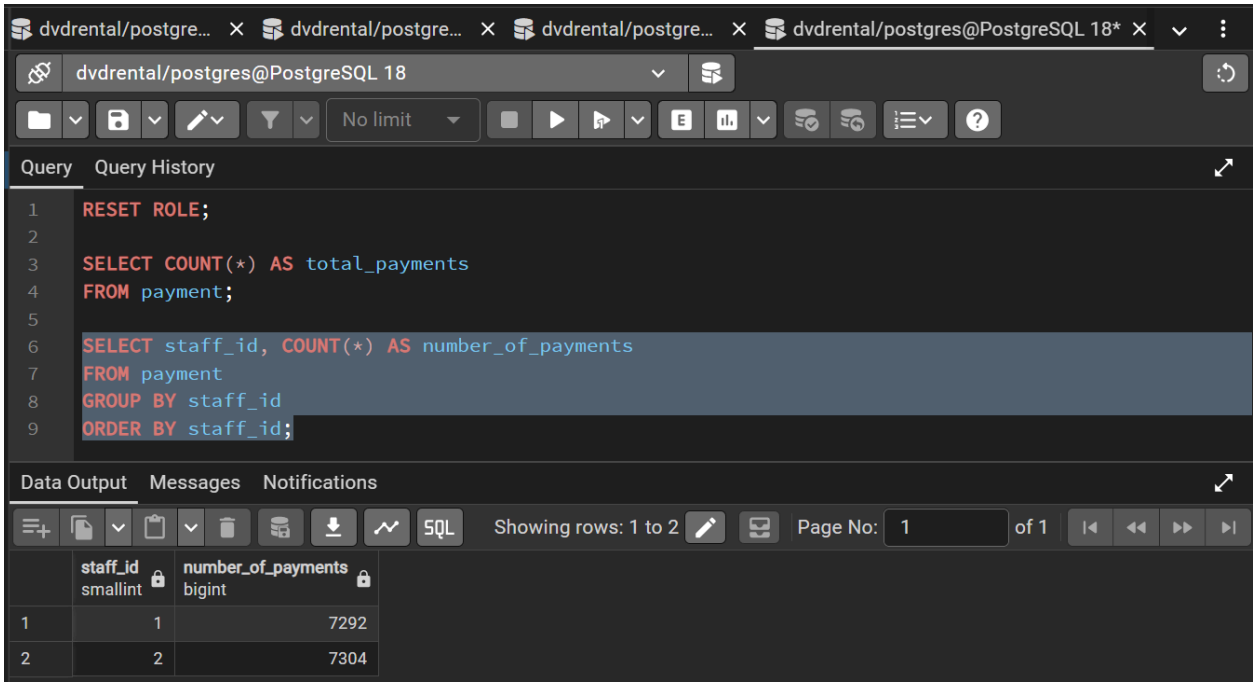


Figure 5. Payment counts by staff_id before applying Row-Level Security.

The result shows:
- staff_id = 1: 7,292 payment records
- staff_id = 2: 7,304 payment records

Without RLS, a cashier with SELECT privilege on the payment table could potentially access payment records belonging to both staff members.

3.2. Enabling Row-Level Security

Row-Level Security was enabled on the payment table:

```
ALTER TABLE payment
ENABLE ROW LEVEL SECURITY;
```

A policy named isolate_cashier_payments was then created:

```
CREATE POLICY isolate_cashier_payments
ON payment
FOR SELECT
TO cashier_role
USING (
   staff_id = (
      SELECT staff_id
      FROM staff
      WHERE LOWER(username) = current_user
   )
);
```

The LOWER(username) function was used because the usernames stored in the staff table are Mike and Jon, while PostgreSQL role names are mike and jon. This ensures that the username comparison is case-insensitive.

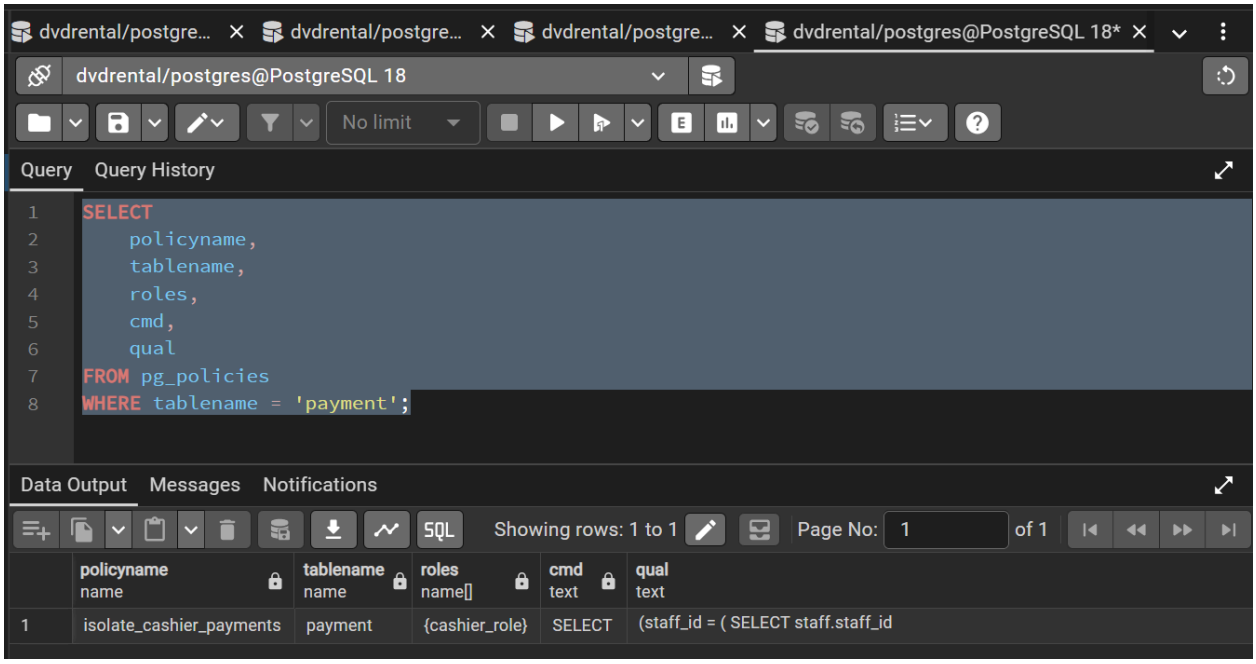


Figure 6. The isolate_cashier_payments RLS policy applied to the payment table. The policy applies only to SELECT operations performed by members of cashier_role.

4. Testing the RLS Policy

4.1. Testing with Mike

The session was switched to the mike role.
SET ROLE mike;

SELECT COUNT(*) AS visible_payments
FROM payment;

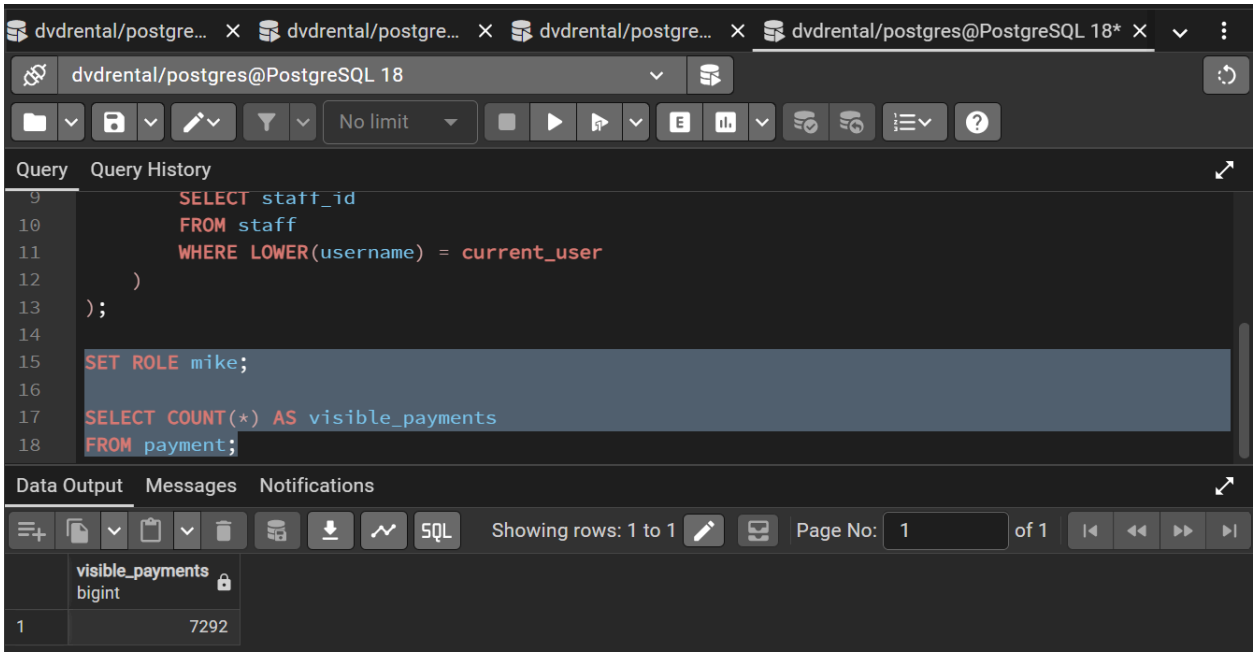


Figure 7. Mike can only access his own 7,292 payment transactions after RLS is enabled.

The result returns exactly 7,292 rows, matching the number of payment records associated with staff_id = 1.

This confirms that Mike can no longer access payment records belonging to Jon.

4.2. Testing with Jon

The same test was performed using the jon role.

RESET ROLE;

SET ROLE jon;

SELECT COUNT(*) AS visible_payments

FROM payment;

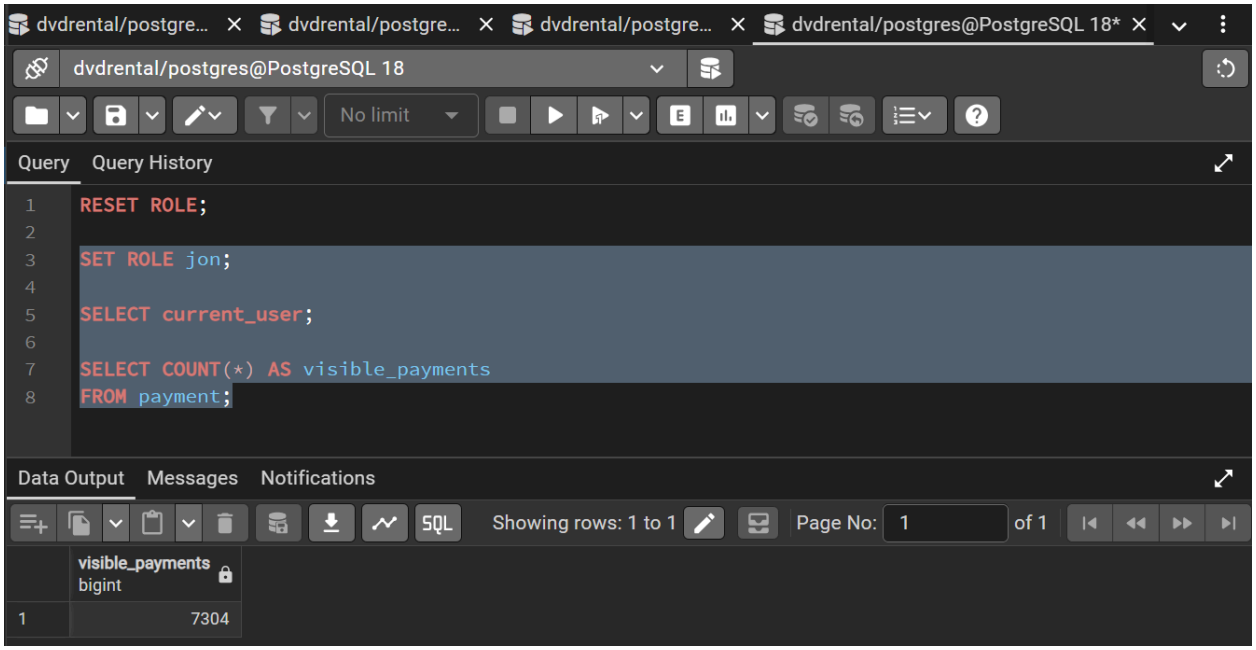
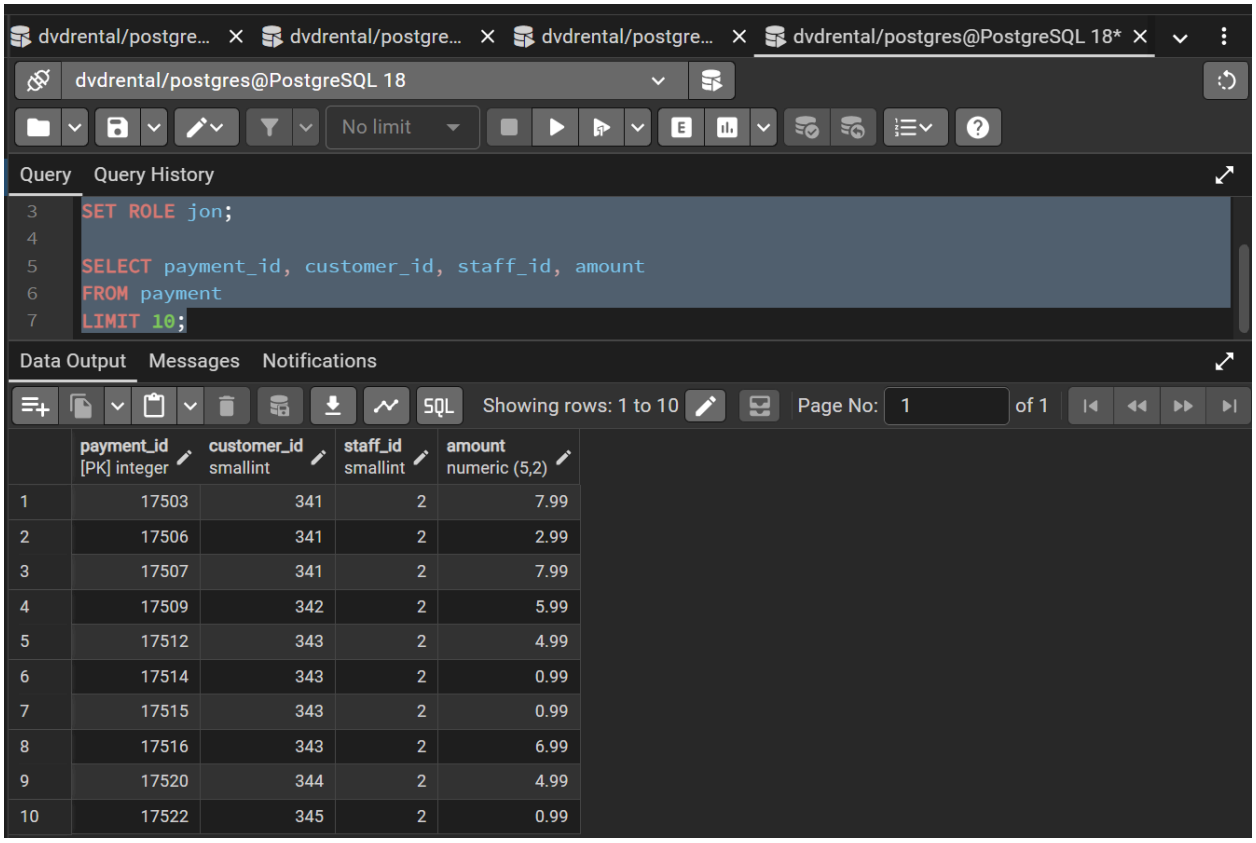


Figure 8. Jon can only access his own 7,304 payment transactions after RLS is enabled.
The query returns exactly 7,304 rows, corresponding to the payments associated with staff_id = 2. Therefore, the RLS policy correctly isolates the payment records between the two cashier accounts.

4.3. Verifying row-level filtering

To visually confirm the effect of the RLS policy, several payment records were retrieved while logged in as Jon.
SET ROLE jon;

SELECT payment_id, customer_id, staff_id, amount
FROM payment
LIMIT 10;



	payment_id [PK] integer	customer_id smallint	staff_id smallint	amount numeric (5,2)
1	17503	341	2	7.99
2	17506	341	2	2.99
3	17507	341	2	7.99
4	17509	342	2	5.99
5	17512	343	2	4.99
6	17514	343	2	0.99
7	17515	343	2	0.99
8	17516	343	2	6.99
9	17520	344	2	4.99
10	17522	345	2	0.99

Figure 9. RLS filters the payment table so that Jon can only view rows with staff_id = 2.

All returned rows contain staff_id = 2, providing direct evidence that PostgreSQL automatically filters the table according to the currently logged-in cashier.

5. Conclusion

This lab successfully implemented two complementary database security mechanisms in PostgreSQL. RBAC was used to control what operations a cashier is allowed to perform. The cashier_role was granted only the required SELECT and INSERT privileges, while unauthorized operations such as DELETE were blocked.

RLS was then used to control which rows a cashier is allowed to access. Although Mike and Jon share the same role and table privileges, the RLS policy ensures that Mike can only access the 7,292 payment records associated with staff_id = 1, while Jon can only access the 7,304 records associated with staff_id = 2.

Therefore, combining RBAC with RLS provides both table-level privilege control and row-level data isolation, significantly improving the security of the dvdrental database.